

TERMINOLOGÍA DE PROGRAMACIÓN USADA

Estructuras de Control

- **for:** Bucle que repite un bloque de código para cada elemento de una secuencia.
- **while:** Bucle que repite un bloque de código mientras una condición lógica sea verdadera.
- **if / elif / else:** Estructuras condicionales que ejecutan bloques de código de forma selectiva según condiciones.
- **break:** Interrumpe la ejecución del bucle actual.
- **continue:** Salta a la siguiente iteración de un bucle.
- **return:** Finaliza una función devolviendo un valor al punto donde fue llamada.

Definición y Uso de Funciones

- **def:** Palabra clave que define una función reutilizable.
- **global:** Palabra clave que permite acceder y modificar variables definidas fuera de la función.
- **None:** Valor especial que indica ausencia de un valor o resultado.

Módulos y Librerías

- **import:** Permite incluir módulos externos para ampliar funcionalidades.
- **pygame:** Biblioteca para el desarrollo de videojuegos.

- **random:** Módulo que genera valores aleatorios.
- **math:** Módulo que proporciona funciones matemáticas avanzadas.

Pygame (Clases y Métodos Relevantes)

- **pygame.Surface:** Clase que representa imágenes y áreas gráficas.
- **pygame.Rect:** Clase que representa rectángulos para posiciones y colisiones.
- **pygame.font.Font:** Clase que gestiona la creación y renderización de texto.
- **pygame.mixer.Sound:** Clase utilizada para cargar y reproducir efectos de sonido.
- **pygame.display.flip():** Método que actualiza la pantalla mostrando los cambios realizados.
- **pygame.time.Clock:** Clase para controlar la velocidad de actualización del juego.
- **pygame.key.get_pressed():** Método que devuelve el estado actual de todas las teclas del teclado.
- **pygame.event.get():** Método que recupera todos los eventos de la cola (como teclas o movimientos de mouse).

Tipos de Datos y Variables

- **list (Lista):** Estructura ordenada y mutable para almacenar múltiples elementos.
- **dict (Diccionario):** Estructura de datos que almacena pares clave-valor.
- **tuple (Tupla):** Secuencia ordenada de elementos inmutable.
- **int:** Tipo de dato numérico entero.
- **float:** Tipo de dato numérico con punto decimal.
- **bool:** Tipo de dato lógico con valores True o False.
- **str:** Cadena de caracteres.

Funciones Matemáticas

- **math.hypot():** Calcula la distancia entre dos puntos en un plano cartesiano.
- **math.cos() / math.sin():** Calculan el coseno y seno de un ángulo.
- **math.pi:** Constante matemática que representa el número pi.

Conceptos de Programación Aplicados

- **Variable local:** Variable definida dentro de una función y usada solo en ella.

- **Variable global:** Variable definida fuera de funciones y accesible desde todo el programa.
- **Alcance de variable (scope):** Define desde dónde se puede acceder a una variable.
- **Estructuras de datos dinámicas:** Colecciones que pueden crecer o disminuir en tiempo de ejecución.
- **Manejo de eventos:** Modelo de programación que responde a acciones del usuario o del sistema.
- **Estados del juego:** Técnica para controlar qué parte del programa se ejecuta (menú, juego, instrucciones, etc.).
- **Patrón Máquina de Estados:** Forma de estructurar el flujo de un videojuego usando variables que indican el estado actual.
- **Modularidad:** Organización del código en funciones independientes y reutilizables.
- **Reutilización de código:** Uso de las mismas funciones para múltiples partes del programa.
- **Orientación a eventos:** Modelo de programación donde el flujo depende de las acciones del usuario o del sistema.

Elementos de Sonido y Gráficos

- **Canal de audio:** Unidad lógica usada por Pygame para reproducir sonidos simultáneos.
- **Superficie:** Área gráfica donde se dibujan elementos.
- **Alpha (transparencia):** Nivel de opacidad de una superficie o imagen.
- **FPS (Frames Per Second):** Velocidad a la que se actualiza la pantalla del juego.

CONCEPTOS DE PROGRAMACIÓN UTILIZADOS EN EL JUEGO ENDZONE

1. PROGRAMACIÓN ESTRUCTURADA

- Uso de funciones: `cargar_imagen_ruta()`, `draw_button()`, `draw_health_bar()`, `draw_brasas()`, `draw_nubes()`, `mostrar_pantalla_info()`, `mostrar_historia()`, `game_loop()`, `start_game()`, `exit_game()`, `show_instructions_screen()`.
- Modularidad: división del código en funciones independientes.
- Reutilización: mismas funciones usadas en varias partes (por ejemplo, barras de vida).

2. VARIABLES Y ALCANCE

- Variables globales: `show_menu`, `run_game`, `show_instructions`.
- Variables locales: `player_pos`, `enemies`, `projectiles` (dentro de funciones).

3. PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

- Uso de clases de Pygame: `pygame.Surface`, `pygame.Rect`, `pygame.font.Font`, `pygame.mixer.Sound`.
- Aunque no se crean clases propias, se aprovechan clases existentes de la librería.

4. ESTRUCTURAS DE DATOS

- Listas: `particles`, `menu_brasas`, `nubes`, `enemies`, `projectiles`, `jefe_projectiles`, `blood_stains`.
- Diccionarios: `jefe_imgs`, `habilidad_nombres`, `historias`.

5. BUCLES Y CONDICIONALES

- Bucle `while`: bucle principal del juego.
- Bucles `for`: recorrido de proyectiles, enemigos, partículas.
- Condicionales `if`: detección de colisiones, cambios de estado, condiciones de victoria/derrota.

6. EVENTOS E INTERACCIÓN

- Manejo de eventos de Pygame: teclado, cierre de ventana.
- Lectura de teclado: `pygame.key.get_pressed()`.

7. MATEMÁTICAS APLICADAS

- Uso de trigonometría: cálculo de vectores normalizados para disparos.
- Colisiones por distancia con `math.hypot()`.

8. ESTADOS DEL JUEGO (MÁQUINA DE ESTADOS)

- Control de pantallas: `show_menu`, `show_instructions`, `run_game`.
- Patrón de diseño básico: máquina de estados simple.

RESUMEN

Este juego utiliza programación estructurada, colecciones de datos, eventos, conceptos de POO a través de Pygame, control de flujo con bucles y condicionales, cálculos matemáticos aplicados y un sistema de estados para controlar el flujo entre menú, instrucciones y juego.